Chapitre 3

Approche proposée pour la génération d'applications web pilotées par BPMN

3.1 Introduction

Les chapitres précédents ont permis de situer le travail dans le domaine de la modélisation des processus métier, de l'ingénierie dirigée par les modèles et de la génération d'applications web. Ils ont également montré que les solutions existantes répondent souvent à une partie du problème : certaines exécutent un processus, d'autres génèrent une interface, exposent une API ou produisent un prototype. La difficulté demeure lorsqu'il faut obtenir une application web complète, maîtrisable par les développeurs et suffisamment liée au modèle de processus pour éviter les écarts entre la conception et l'exécution.

Le présent chapitre expose l'approche proposée pour répondre à cette difficulté. Elle s'appuie sur le framework ProOWeb, conçu comme un environnement d'ingénierie permettant de produire des applications web orientées processus à partir d'un modèle BPMN et d'une spécification complémentaire. L'objectif n'est pas de transformer directement un diagramme en code sans contrôle, mais de mettre en place une chaîne de génération organisée, vérifiable et compatible avec l'évolution du code produit.

L'approche proposée repose sur trois idées directrices. Le modèle BPMN conserve le rôle de description du workflow. Une spécification enrichie, appelée *spec-v1*, complète les informations qui ne sont pas portées par le diagramme, notamment les rôles, les données, les affectations, les règles de visibilité et les comportements d'entrée-sortie. Enfin, un modèle pivot permet de stabiliser les informations avant la génération des artefacts logiciels. Cette organisation rapproche ProOWeb d'une démarche d'ingénierie dirigée par les modèles, tout en l'adaptant aux contraintes pratiques du développement web.

Ce chapitre présente d'abord la vue d'ensemble de l'approche et son positionnement

dans le cycle de la Model-Driven Architecture. Il décrit ensuite l'architecture globale du framework proposé, le modèle d'entrée enrichi, les règles de transformation et la génération des artefacts. La dernière partie traite de la préservation du code personnalisé et de la synchronisation, car une application générée ne peut être réellement utile que si elle peut évoluer sans détruire les adaptations réalisées par les développeurs.

3.2 Vue d'ensemble de l'approche

L'approche proposée vise à construire une application web orientée processus en conservant le modèle métier comme point d'entrée. ProOWeb ne se limite donc pas à une plateforme d'exécution de workflows. Il s'agit plutôt d'un framework de génération qui produit un projet applicatif complet, composé d'un back-end Spring Boot, d'un front-end React/Vite, d'une organisation de persistance, de scripts de déploiement, de profils d'exécution et de mécanismes de sécurité. Ce choix correspond à la position du projet : offrir un gain d'automatisation sans enfermer l'équipe dans une plateforme propriétaire où le code généré serait difficile à reprendre.

Dans cette perspective, l'application web produite reste un projet logiciel. Elle peut être versionnée, testée, modifiée, relue et déployée selon les pratiques classiques d'une équipe de développement. ProOWeb intervient principalement pour réduire la traduction manuelle du processus vers les composants techniques répétitifs : routes, catalogues de processus, listes de tâches, contrats d'exécution, métadonnées de formulaires, règles d'affectation, vues de suivi et structures de données. Le développeur n'est pas supprimé de la chaîne ; son effort est déplacé vers la validation du modèle, l'enrichissement, les adaptations métier et l'évolution maîtrisée du code.

Le principe général peut être résumé ainsi : le modèle BPMN décrit le déroulement du travail ; la spécification *spec-v1* décrit ce qui est nécessaire à l'application ; le modèle pivot stabilise les informations ; les transformations produisent des artefacts back-end et front-end. Ce découpage est essentiel, car BPMN n'exprime pas naturellement toutes les informations requises par une application web. Une tâche BPMN peut indiquer qu'un utilisateur doit réaliser une activité, mais elle ne dit pas toujours quels champs doivent être affichés, quelles données doivent être lues, quels résultats doivent être sauvegardés, ni quels rôles peuvent voir ou traiter cette tâche.

3.2.1 Positionnement de l'approche dans le cycle MDA

La Model-Driven Architecture, ou MDA, organise le développement autour de plusieurs niveaux d'abstraction : le modèle indépendant du calcul, le modèle indépendant de la plateforme, le modèle spécifique à la plateforme et le code. Cette séparation est utile pour ProOWeb, car elle permet de comprendre comment un processus métier peut deve-

nir progressivement une application exécutable sans passer directement du diagramme au code source [40].

Dans l'approche proposée, la MDA n'est pas reprise comme un cadre abstrait détaché du projet. Elle sert de repère méthodologique pour distinguer les responsabilités. Le modèle de processus exprime d'abord l'organisation métier. La spécification enrichie ajoute les éléments applicatifs nécessaires. Les contrats intermédiaires structurent les informations dans une forme exploitable par le générateur. Le code produit matérialise ces décisions dans une architecture concrète. Cette progression évite de concentrer toute la complexité dans les templates de génération.

Niveau PIM : modélisation du processus métier

Au niveau PIM, c'est-à-dire *Platform Independent Model*, l'application n'est pas encore attachée à une technologie précise. Le modèle BPMN décrit les activités, les événements, les flux, les décisions et les participants. Il représente le fonctionnement du processus indépendamment du framework web cible. Dans ProOWeb, ce modèle est conservé comme source principale du workflow.

Le PIM n'est cependant pas réduit au fichier BPMN brut. Pour que le modèle puisse conduire vers une application, il doit être complété par une description des besoins applicatifs. C'est le rôle de *spec-v1*. Cette spécification reste indépendante de l'implémentation technique, mais elle apporte des informations précises sur les activités, les affectations, les entrées, les sorties, la visibilité et les données partagées. Elle reste donc au niveau de la conception applicative, sans encore imposer la forme exacte des classes Java ou des composants React.

Cette articulation permet de préserver la lisibilité du processus. Le BPMN demeure compréhensible par les acteurs qui raisonnent sur le workflow. La spécification enrichie reçoit les informations supplémentaires qui rendraient le diagramme trop chargé si elles étaient représentées graphiquement. La séparation est importante : elle évite de détourner BPMN de sa fonction principale tout en conservant les informations nécessaires à la génération.

Niveau PSM : modèle applicatif intermédiaire

Le niveau PSM, ou *Platform Specific Model*, rapproche progressivement la description du système des choix techniques. Dans ProOWeb, ce rôle est assuré par les contrats intermédiaires, notamment le *runtime contract*, le *data contract*, le catalogue de formulaires et le registre des processus générés. Ces contrats ne sont pas encore du code au sens strict ; ils structurent les informations de façon suffisamment précise pour que les générateurs puissent produire des fichiers cohérents.

Le *runtime contract* décrit le comportement exécutable du processus déployé : acti-

vités, transitions, rôles autorisés à démarrer une instance, activités de départ, rôles de supervision, affectations et règles de visibilité. Le *data contract* décrit les sources d'entrée, les mappings de sortie, la stratégie de stockage et le lignage des données. Le catalogue de formulaires expose les métadonnées utilisées par le front-end pour construire les vues de traitement. Le registre généré permet à l'application cible de connaître les processus et versions disponibles.

Ce niveau intermédiaire joue un rôle de sécurité conceptuelle. Au lieu de laisser chaque template interpréter directement le BPMN et la spécification, ProOWeb transforme d'abord ces éléments en contrats stabilisés. Le générateur travaille alors sur une représentation normalisée. Cette décision rend la chaîne plus testable, car les erreurs peuvent être détectées avant l'écriture du code.

Niveau code : génération des artefacts logiciels

Le dernier niveau correspond à la production des artefacts logiciels. Dans la version étudiée de ProOWeb, ces artefacts sont organisés autour d'un back-end Spring Boot, d'un front-end React/Vite, d'une persistance relationnelle accompagnée par Liquibase, de scripts de build et de déploiement, ainsi que de profils d'exécution tels que *dev*, *demo*, *test*, *preprod* et *prod*.

La génération ne produit pas seulement des fichiers isolés. Elle construit une application organisée autour de plusieurs modules : identité, sécurité, organisation, processus, données, notifications, supervision et audit. Elle produit aussi des composants liés aux processus déployés, notamment les contrats d'exécution, les catalogues de processus, les modules de runtime, les API de démarrage et de traitement des tâches, les vues de travail côté front-end et les métadonnées nécessaires au filtrage selon les rôles.

Ce passage au code matérialise les décisions précédentes. Une activité manuelle devient une tâche visible et traitable par un utilisateur. Une règle d'affectation devient une politique de résolution des candidats. Une donnée partagée devient une structure exploitable par l'application. Une règle de visibilité devient un filtre appliqué lors de la consultation des tâches ou des instances. Le code généré reste toutefois suivi par ProOWeb à l'aide d'un manifeste de fichiers gérés, afin de permettre des régénérations contrôlées.

3.2.2 Principe du double modèle d'entrée

La génération repose sur un double modèle d'entrée. Ce choix est au centre de l'approche, car il répond à la limite principale de BPMN dans la génération d'applications web. Le diagramme de processus porte la logique de workflow, mais il ne suffit pas à définir l'ensemble des composants applicatifs. À l'inverse, une spécification purement technique pourrait produire du code, mais elle perdrait le lien direct avec le processus métier. ProOWeb combine donc les deux sources.

Le double modèle d'entrée permet de séparer les préoccupations sans les isoler. Le BPMN fournit l'ordre des activités, les transitions et les éléments structurants du processus. La spécification *spec-v1* fournit les détails applicatifs. Le modèle pivot réconcilie les deux pour produire une représentation cohérente et exploitable par la génération.

Modèle BPMN comme description du workflow

Le modèle BPMN constitue la description principale du workflow. Il donne la structure du processus : tâches, événements, flux de séquence, passerelles et responsabilités. Dans ProOWeb, les activités présentes dans le fichier BPMN sont utilisées comme références pour la spécification. La validation vérifie que les activités enrichies dans *spec-v1* correspondent bien à des identifiants présents dans le modèle BPMN.

Cette contrainte évite une situation fréquente dans les projets manuels : le diagramme évolue d'un côté, tandis que l'application continue à exécuter une logique ancienne. En reliant explicitement les activités de la spécification aux identifiants BPMN, ProOWeb maintient un lien entre le processus modélisé et les artefacts produits. Le modèle n'est donc pas seulement une documentation, il devient une source contrôlée de génération.

Le BPMN est également utilisé pour construire la logique de flux. Lors de la génération du contrat d'exécution, ProOWeb extrait les transitions entre activités et organise les liens entrants et sortants. Ces informations servent ensuite à déterminer les activités de départ, les transitions disponibles et les chemins possibles lors de l'exécution d'une instance.

Spécification enrichie comme complément applicatif

La spécification *spec-v1* complète le BPMN avec les informations nécessaires à l'application. Elle contient notamment le type des activités, les rôles candidats, les règles d'affectation, les sources d'entrée, les mappings de sortie, la stratégie de stockage, les règles de visibilité et les données partagées. Elle précise également le comportement des activités automatiques à travers des références de gestionnaires, des types de tâches et des configurations typées.

Ce complément est indispensable. Une activité BPMN peut être manuelle ou automatique selon son type dans le diagramme, mais l'application a besoin de savoir comment l'activité sera affectée, par qui elle sera visible, quelles données elle consomme et ce qu'elle produit. Une tâche de validation, par exemple, n'est pas seulement un rectangle dans un diagramme : elle devient une vue utilisateur, une action contrôlée, une écriture de données, une transition et une trace d'audit.

La spécification enrichie ne doit donc pas être considérée comme un ajout secondaire. Elle constitue le pont entre le processus et l'application. Elle rend explicites des décisions qui seraient autrement dispersées dans le code source.

Rôle du modèle pivot dans la génération

Le modèle pivot désigne l'ensemble des contrats intermédiaires construits à partir du BPMN et de *spec-v1*. Son rôle est de stabiliser les données avant leur projection vers le code. Il évite que la génération soit une succession de règles dispersées directement dans les templates.

Dans ProOWeb, ce modèle pivot prend plusieurs formes complémentaires. Le *runtime contract* décrit l'exécution. Le *data contract* décrit les données et leur circulation. Les catalogues front-end exposent les informations nécessaires aux composants React. Les registres générés décrivent les processus disponibles et leurs versions. Le modèle pivot joue donc un rôle de médiation : il n'est ni le modèle source ni le code final, mais il rend le passage de l'un à l'autre plus déterministe.

Le tableau suivant synthétise les responsabilités de chaque niveau.

TABLE 3.1 – Rôle des principaux artefacts dans l'approche ProOWeb

Artefact	**Rôle principal**	**Contribution à la génération**
Modèle BPMN	Décrit le workflow, les activités, les événements, les flux et les décisions.	Sert de référence au processus et fournit la structure de navigation métier.
Spécification *spec-v1*	Complète le processus par les données, rôles, affectations, entrées, sorties et règles de visibilité.	Rend le BPMN exploitable pour produire une application web.
Modèle pivot	Normalise les informations issues du BPMN et de la spécification.	Stabilise la génération à travers des contrats intermédiaires.
Artefacts générés	Matérialisent l'application en code source back-end, front-end, configuration et scripts.	Produisent une application web exploitable, versionnable et personnalisable.

3.3 Architecture globale du framework proposé

L'architecture globale de ProOWeb est organisée autour de deux espaces complémentaires. Le premier est l'environnement d'édition et de génération. Il permet de configurer le projet, de modéliser les processus, de rédiger la spécification enrichie, de valider les informations et de déclencher la génération. Le second est l'application web cible produite par le framework. Elle comprend un back-end, un front-end, une organisation de persistance,

des profils de déploiement, des mécanismes de sécurité et des modules liés à l'exécution des processus.

Cette séparation est importante. ProOWeb n'est pas seulement une application générée ; c'est d'abord un outil qui produit d'autres applications. Le code de l'éditeur est isolé dans le répertoire `ProOWeb/`, tandis que le projet généré est écrit à la racine du dépôt, notamment dans `src/backend/springboot`, `src/frontend/web/react` et `deployment/`. Cette organisation permet de distinguer le framework de génération et l'application cible.

3.3.1 Architecture fonctionnelle du framework

L'architecture fonctionnelle du framework correspond à la chaîne de traitement qui va du modèle au code. Elle comporte un sous-système d'ingestion du BPMN, un module d'enrichissement, un noyau de transformation modèle-à-modèle et un moteur de génération modèle-à-texte. À ces composants s'ajoutent des fonctions transversales : validation, gestion des versions, déploiement des processus, migration intelligente, gouvernance des templates et production de rapports.

Cette architecture suit une logique de responsabilité séparée. Chaque module traite un problème précis. L'ingestion ne doit pas décider des écrans à produire. L'enrichissement ne doit pas écrire les fichiers Java ou JavaScript. Le noyau de transformation construit les contrats, tandis que le moteur de génération applique les templates. Ce découpage rend l'approche plus contrôlable et permet de faire évoluer une partie du framework sans réécrire l'ensemble.

Sous-système d'ingestion du modèle BPMN

Le sous-système d'ingestion est chargé de lire les modèles BPMN et d'extraire les éléments utiles à la génération. Dans ProOWeb, les modèles sont stockés dans l'espace de travail de l'éditeur, sous la forme de versions associées à un modèle de processus. Le cycle de vie distingue notamment les états *DRAFT*, *VALIDATED*, *DEPLOYED* et *RETIRED*. Cette gestion des versions évite qu'un processus encore en cours de modification soit traité comme un processus prêt pour la génération.

L'ingestion ne se limite pas à la lecture du XML. Elle vérifie les identifiants d'activités, la cohérence des éléments manipulés et la relation entre le modèle graphique et l'éditeur XML. Le projet prévoit une synchronisation bidirectionnelle entre le canvas BPMN et l'éditeur XML, ce qui permet à l'utilisateur de travailler soit de manière graphique, soit de manière textuelle. Le recours à Monaco pour l'édition apporte aussi des marqueurs de vérification et une meilleure maîtrise du contenu.

Le sous-système d'ingestion prépare donc une représentation exploitable du workflow. Cette représentation n'est pas encore suffisante pour générer toute l'application, mais elle constitue la base à laquelle seront rattachées les informations de *spec-v1*.

Module d'enrichissement à partir de la spécification

Le module d'enrichissement associe la spécification *spec-v1* aux activités détectées dans le BPMN. Il vérifie que les activités décrites dans la spécification existent bien dans le modèle. Il contrôle aussi les valeurs autorisées pour les types d'activités, les modes d'affectation, les stratégies, les sources d'entrée, les stratégies de stockage et les types de champs partagés.

L'enrichissement est un moment critique. C'est à cette étape que l'on passe d'un workflow lisible à un processus exploitable par une application. La spécification indique par exemple si une activité est `MANUAL` ou `AUTOMATIC`, quels rôles peuvent la traiter, quelles données doivent être lues, quels résultats doivent être conservés et quels rôles peuvent consulter les données produites. Sans cette étape, le générateur serait obligé de déduire des informations absentes du modèle, ce qui rendrait le résultat fragile.

Le module d'enrichissement sert aussi à préparer la sécurité fonctionnelle. Les rôles candidats, les rôles de supervision et les règles de visibilité sont déclarés avant la génération. Ils peuvent donc être repris par le back-end et le front-end, au lieu d'être ajoutés manuellement après coup.

Noyau de transformation modèle-à-modèle

Le noyau de transformation modèle-à-modèle construit les contrats intermédiaires. Il prend en entrée le modèle BPMN et la spécification enrichie, puis produit des représentations normalisées. Ces contrats correspondent au modèle pivot de ProOWeb. Ils contiennent les informations nécessaires au runtime, aux données, aux formulaires et aux catalogues.

Le *runtime contract* est généré à partir de la version du processus déployé. Il contient le numéro de version, le statut, les rôles autorisés à démarrer le processus, les rôles de supervision, les activités, les transitions et les informations de flux. Un résumé indique notamment le nombre d'activités, le nombre d'activités manuelles et automatiques, le nombre de transitions et les rôles concernés.

Le *data contract* organise les sources d'entrée et les mappings de sortie. Il construit également des informations de lignage, afin de savoir comment les données circulent entre le contexte du processus, les activités précédentes, les données partagées, les services back-end ou les services externes. Cette transformation rend la gestion des données plus explicite.

Moteur de génération modèle-à-texte

Le moteur de génération modèle-à-texte produit les fichiers de l'application. Il applique des templates organisés par technologie et par module. La génération back-end utilise des templates Spring Boot. La génération front-end utilise des templates React. Les fichiers

de déploiement sont produits à partir de templates Docker et de scripts.

Le moteur ne doit pas porter seul toute la logique de décision. Les templates doivent principalement matérialiser des décisions déjà stabilisées dans les contrats. Cette organisation évite que le générateur devienne un ensemble difficile à maintenir de conditions dispersées. Elle facilite aussi la gouvernance des personnalisations, car les fichiers produits sont suivis dans le manifeste et peuvent être soumis à des règles d'override.

Chaque fichier généré reçoit des métadonnées d'appartenance, notamment les propriétaires fonctionnels et la catégorie du fichier. Ces informations sont ensuite utilisées dans le manifeste `.prooweb-managed.json`. Grâce à ce suivi, la migration intelligente peut comparer les fichiers existants, détecter les changements manuels et produire un rapport détaillé.

3.3.2 Architecture de l'application web cible

L'application web cible est produite comme un projet complet. Elle ne se limite pas à l'exécution d'un diagramme. Elle comprend une interface web, des services back-end, une persistance, une sécurité, des modules de gestion des processus, des tests, des scripts et des profils de déploiement. Cette organisation vise à obtenir une base applicative exploitable dès la génération initiale.

Le choix d'une application source-owned est central. Le code appartient à l'équipe de développement. Il peut être lu et modifié, contrairement à certaines plateformes où la logique reste enfermée dans un environnement propriétaire. ProOWeb se positionne ainsi entre le développement manuel, qui donne un contrôle maximal mais demande beaucoup de temps, et les BPMS classiques, qui accélèrent la modélisation mais réduisent parfois la maîtrise du code généré.

Front-end généré

Le front-end généré repose sur React et Vite. Il contient les composants nécessaires à l'interaction avec l'application, notamment les vues de processus, les listes de tâches, les tableaux de bord, les actions d'affectation, les vues d'instances et les consoles de supervision. Lorsque les processus sont déployés, ProOWeb produit aussi des modules spécifiques tels que le registre des processus générés, les catalogues de tâches et les contrats front-end associés à chaque version.

La génération front-end prend en compte les métadonnées des formulaires et les règles de visibilité. L'interface peut ainsi afficher les tâches à traiter, les tâches à affecter, les instances visibles et les opérations disponibles selon les rôles de l'utilisateur. Elle ne constitue toutefois pas la seule couche de sécurité. Les restrictions affichées côté client améliorent l'expérience utilisateur, mais le contrôle final doit rester appliqué côté back-end.

Le front-end généré contient également des éléments de travail pour les opérations de

runtime. Le *Process Runtime Workbench* donne accès aux processus déployés, au démarrage d'instances, au traitement des tâches, à l'affectation, à la consultation des timelines et aux opérations de supervision lorsque l'utilisateur possède les rôles requis.

Back-end généré

Le back-end généré repose sur Spring Boot. Sa structure suit une organisation modulaire stricte, proche d'une architecture hexagonale. Les modules sont séparés en couches domaine, application et infrastructure lorsque cela est pertinent. Le projet généré contient notamment des modules comme `gateway`, `kernel`, `common`, `system`, `identity`, `organization` et un module de composition `prooweb-application`.

Cette organisation vise à éviter un back-end monolithique sans séparation interne. Le module `gateway` joue le rôle de point d'entrée pour les API. Les modules de domaine portent les objets métier. Les modules d'application contiennent les cas d'utilisation. Les modules d'infrastructure assurent l'adaptation vers la persistance, les configurations ou les services externes. Cette séparation rend le code généré plus lisible et plus maintenable.

Pour les processus déployés, le back-end génère des ressources de runtime, des classes de domaine liées aux instances, aux tâches et aux états, des services d'orchestration, des adaptateurs de stockage et un contrôleur d'API. Les endpoints produits couvrent notamment les options de démarrage, le démarrage d'une instance, la liste des tâches, l'affectation, la complétion, la consultation d'une instance, la timeline, l'arrêt et l'archivage.

Organisation en couches

L'organisation en couches vise à séparer les responsabilités. La couche d'interface reçoit les requêtes. La couche application orchestre les cas d'utilisation. La couche domaine porte les règles centrales. La couche infrastructure connecte le système aux technologies externes. Dans ProOWeb, cette séparation n'est pas seulement une préférence architecturale ; elle permet aussi de mieux préserver le code personnalisé.

Si un développeur doit ajouter une règle métier, il peut le faire dans un point d'extension ou dans une couche prévue à cet effet, sans modifier directement un template de génération. Si une API doit être adaptée, elle peut être traitée dans le module approprié. Cette discipline facilite la régénération, car les fichiers générés et les ajouts manuels peuvent être mieux distingués.

L'application cible est également accompagnée de tests. Le projet prévoit des tests unitaires, des tests d'intégration, des tests de runtime et des rapports de couverture. Cette présence des tests est importante dans une approche générative : elle permet de vérifier non seulement que le code compile, mais aussi que les comportements essentiels restent valides après génération, migration ou reconfiguration.

Persistance et gestion des données

La persistance est organisée autour d'une base relationnelle et d'une discipline Liquibase lorsque le pack correspondant est activé. ProOWeb peut générer des changelogs, notamment un fichier maître et des changesets de base. Les processus déployés peuvent également conduire à des structures liées aux données partagées.

Les données manipulées par le runtime ne sont pas toutes de même nature. Certaines appartiennent à l'instance de processus. D'autres relèvent du modèle de données partagé. La spécification *spec-v1* permet de définir des stratégies de stockage : `INSTANCE`, `SHARED` ou `BOTH`. Cette distinction est importante pour éviter de mélanger le contexte temporaire d'une instance et les données persistantes réutilisables.

La génération du *data contract* permet aussi de conserver un lignage des données. Les mappings d'entrée et de sortie peuvent indiquer comment une activité consomme des informations provenant du contexte, d'une activité précédente ou d'un espace partagé, puis comment elle écrit ses résultats. Cette traçabilité facilite la compréhension de l'application et prépare les contrôles sur la visibilité des données.

TABLE 3.2 – Synthèse de l'architecture cible générée par ProOWeb

Composant	**Organisation générée**	**Rôle dans l'application cible**
Front-end	React/Vite, modules de processus, workbench runtime, catalogues générés.	Fournir les vues de traitement, de suivi, d'affectation et de supervision.
Back-end	Spring Boot, structure modulaire stricte, séparation gateway, domaine, application et infrastructure.	Exposer les API, appliquer les règles, gérer les processus, la sécurité et les données.
Persistance	Base relationnelle, Liquibase, ressources de runtime et données partagées.	Conserver les utilisateurs, rôles, instances, tâches, traces et données métier.
Déploiement	Docker, scripts, profils *dev*, *demo*, *test*, *preprod*, *prod*.	Faciliter l'exécution selon les environnements du projet.
Gouvernance	Manifeste des fichiers gérés, migration intelligente et overrides de templates.	Préserver le code personnalisé et rendre la régénération contrôlable.

3.4 Modèle d'entrée enrichi : spécification *spec-v1*

La spécification *spec-v1* constitue le complément applicatif du modèle BPMN. Elle formalise les informations nécessaires pour transformer un workflow en application web. Son rôle est d'éviter que le générateur interprète librement le diagramme. Les décisions importantes sont déclarées dans une structure explicite, puis vérifiées avant génération.

Cette spécification repose sur une version de schéma fixée à 1. Elle organise les activités, les règles de démarrage, les rôles de supervision, les données partagées, les entrées, les sorties, les stratégies d'affectation et les règles de visibilité. Elle est stockée avec les versions de modèles de processus et participe au cycle de vie de validation, simulation, promotion et déploiement.

3.4.1 Structure générale de la spécification

La structure générale de *spec-v1* s'organise autour de blocs fonctionnels. Le bloc de démarrage indique les rôles autorisés à lancer une instance et les activités de départ. Le bloc des moniteurs précise les rôles capables de superviser. Le bloc des activités décrit chaque étape du processus. Le bloc des données partagées définit les entités, champs et relations manipulés au-delà d'une instance unique.

Chaque activité est décrite par un ensemble de métadonnées. Ces informations comprennent le type d'activité, les rôles candidats, les règles d'affectation, les sources d'entrée, les sorties, la visibilité et, pour les activités automatiques, la configuration du gestionnaire à exécuter. Cette structure rend la spécification suffisamment riche pour alimenter la génération du runtime et du front-end.

La spécification utilise aussi des valeurs contrôlées. Les types d'activités sont par exemple `MANUAL` ou `AUTOMATIC`. Les modes d'affectation sont `AUTOMATIC` ou `MANUAL`. Les stratégies d'affectation appartiennent à une liste définie, comme `ROLE_QUEUE`, `SUPERVISOR_ONLY`, `SUPERVISOR_THEN_ANCESTORS`, `UNIT_MEMBERS`, `SINGLE_MATCH_ONLY`, `MANUAL_ONLY` ou `ROUND_ROBIN`. Ce contrôle limite les erreurs et rend la génération plus prévisible.

3.4.2 Métadonnées des activités

Les métadonnées des activités précisent la manière dont chaque étape BPMN doit être prise en charge dans l'application. Par défaut, ProOWeb peut construire une spécification initiale à partir des activités détectées dans le modèle BPMN. Les tâches correspondant à des éléments automatiques du BPMN, comme les tâches de service, les scripts, les règles métier ou les tâches d'envoi et de réception, peuvent être considérées comme automatiques. Les autres activités sont traitées comme manuelles, sauf précision contraire.

Une activité manuelle est destinée à être traitée par un utilisateur. Elle doit donc

être liée à des rôles candidats, à une stratégie d'affectation, à des données visibles et à des sorties attendues. Une activité automatique est exécutée par le système. Elle dispose alors d'une référence de gestionnaire, d'un type de tâche automatique, d'un mode de déclenchement et d'une configuration.

La spécification distingue les modes de déclenchement des activités automatiques. Une activité peut être déclenchée manuellement, immédiatement ou de manière différée. Elle peut aussi s'appuyer sur des types de tâches automatiques réutilisables, comme l'envoi d'email, la diffusion de message, la génération de document, le nettoyage de données, la transformation, l'appel webhook, l'audit ou le délai. Cette organisation évite de recoder plusieurs fois des comportements techniques semblables.

3.4.3 Affectation des tâches et sécurité

L'affectation des tâches est un élément central dans une application orientée processus. Une tâche manuelle ne doit pas seulement exister ; elle doit être attribuée ou rendue disponible aux bons utilisateurs. ProOWeb formalise cette dimension dans la spécification à travers le mode d'affectation, la stratégie, les rôles candidats, les rôles autorisés à affecter manuellement et la règle `allowPreviouslyAssignedAssignee`.

La stratégie `ROLE_QUEUE` permet de placer une tâche dans une file accessible aux utilisateurs possédant les rôles candidats. `SUPERVISOR_ONLY` et `SUPERVISOR_THEN_ANCESTORS` exploitent la hiérarchie organisationnelle. `UNIT_MEMBERS` s'appuie sur les membres d'une unité. `SINGLE_MATCH_ONLY` exige une résolution non ambiguë. `MANUAL_ONLY` laisse l'affectation à un acteur habilité, tandis que `ROUND_ROBIN` organise une répartition déterministe.

Ces règles sont liées à la sécurité. L'autorisation dans ProOWeb repose sur un RBAC interne, c'est-à-dire un contrôle d'accès fondé sur les rôles. L'authentification peut être interne ou déléguée à un fournisseur IAM externe, mais l'autorisation reste maîtrisée par l'application générée. Cette séparation est importante : un utilisateur peut être reconnu par un fournisseur externe sans pour autant posséder les droits applicatifs nécessaires au traitement d'une tâche.

Le tableau suivant présente les principales stratégies d'affectation.

TABLE 3.3 – Stratégies d'affectation prises en charge dans la spécification

Stratégie	**Interprétation dans ProOWeb**
`ROLE_QUEUE`	Rend la tâche accessible aux utilisateurs possédant les rôles candidats déclarés.
`SUPERVISOR_ONLY`	Oriente la tâche vers le superviseur associé au contexte organisationnel.
`SUPERVISOR_THEN_ANCESTORS`	Recherche un superviseur, puis remonte la hiérarchie si nécessaire.
`UNIT_MEMBERS`	Affecte la tâche aux membres d'une unité organisationnelle.
`SINGLE_MATCH_ONLY`	N'accepte l'affectation automatique que si un seul candidat correspond.
`MANUAL_ONLY`	Exige une action explicite d'un utilisateur autorisé à affecter.
`ROUND_ROBIN`	Répartit les tâches de manière déterministe entre plusieurs candidats.

3.4.4 Gestion des entrées et sorties de données

La gestion des entrées et sorties transforme une activité en opération applicative. Une activité peut lire des données provenant du contexte du processus, d'une activité précédente, d'un espace de données partagées, d'un service back-end ou d'un service externe. Ces sources sont représentées par des types contrôlés : `PROCESS_CONTEXT`, `PREVIOUS_ACTIVITY`, `SHARED_DATA`, `BACKEND_SERVICE` et `EXTERNAL_SERVICE`.

Chaque source d'entrée peut posséder des mappings. Ces mappings indiquent comment une information de départ est projetée vers une donnée attendue par l'activité. À la sortie, l'activité peut produire des mappings qui indiquent comment les résultats seront sauvegardés. La stratégie de stockage précise si la sortie est conservée dans l'instance, dans les données partagées ou dans les deux.

Cette organisation évite que les données soient traitées comme des variables floues du workflow. Elle permet de construire un *data contract*, de produire un lignage et de savoir quelles activités consomment ou produisent des entités partagées. Dans une application où plusieurs processus peuvent manipuler des données communes, cette précision devient indispensable.

3.4.5 Règles de visibilité

Les règles de visibilité déterminent qui peut voir une activité et qui peut consulter les données associées. ProOWeb distingue notamment `activityViewerRoles` et `dataViewerRoles`. Cette séparation est utile, car un utilisateur peut avoir le droit de savoir qu'une tâche existe sans nécessairement accéder à toutes les données qu'elle manipule.

La visibilité est appliquée au runtime. Les APIs de consultation des tâches et des instances acceptent les rôles de l'utilisateur afin de filtrer les réponses. Le front-end peut ainsi afficher les informations autorisées sans exposer inutilement les détails sensibles. La logique de sécurité ne repose pas uniquement sur l'interface : le back-end reste responsable de l'application stricte des règles.

Ces règles préparent aussi la supervision. Les rôles de type `PROCESS_MONITOR` ou `ADMINISTRATOR` peuvent accéder à des vues plus larges, comme l'audit des actions de gouvernance, l'arrêt ou l'archivage d'une instance, ou encore l'affectation forcée. Ces opérations sont sensibles et doivent donc être distinguées des actions ordinaires de traitement.

3.4.6 Modèle de données partagé

Le modèle de données partagé décrit les entités qui peuvent être produites ou consommées par plusieurs activités, voire par plusieurs processus. Une entité partagée possède une clé, un nom de table, des champs, des types, des contraintes et éventuellement des relations. Les types de champs reconnus incluent notamment `STRING`, `TEXT`, `INTEGER`, `LONG`, `DECIMAL`, `BOOLEAN`, `DATE`, `DATETIME`, `JSON` et `UUID`. Les relations peuvent être de type `MANY_TO_ONE`, `ONE_TO_MANY`, `ONE_TO_ONE` ou `MANY_TO_MANY`.

La présence d'un modèle partagé permet de dépasser une vision purement locale du processus. Certaines données ne doivent pas disparaître à la fin d'une instance. Elles doivent être persistées, consultées et parfois réutilisées. ProOWeb peut alors générer des ressources back-end liées à ces entités et produire des informations de catalogue côté front-end.

La spécification prévoit aussi des contrôles de cohérence : noms de champs valides, clés d'entités normalisées, absence de doublons, validité des noms de tables, cohérence des relations et existence des entités cibles. Ces contrôles montrent que le modèle partagé n'est pas une simple note descriptive. Il participe à la génération de structures persistantes.

3.4.7 Correspondance entre BPMN, spécification et artefacts générés

La force de ProOWeb réside dans la correspondance maintenue entre le BPMN, la spécification et les artefacts générés. Une activité BPMN est enrichie dans *spec-v1*. Elle

est projetée dans le *runtime contract*. Elle devient ensuite une ou plusieurs parties de l'application : tâche, action, route, composant front-end, règle de visibilité, entrée dans un catalogue ou trace d'audit.

Cette correspondance assure la traçabilité. Lorsqu'un développeur observe un artefact généré, il doit pouvoir retrouver la source conceptuelle qui l'a justifié. Lorsqu'un analyste modifie une activité ou une règle d'affectation, le changement peut être répercuté dans les contrats et dans les fichiers générés. Cette traçabilité ne supprime pas le travail d'adaptation, mais elle limite les écarts invisibles entre le modèle et l'application.

TABLE 3.4 : Correspondance entre les informations d'entrée et les artefacts générés

Information d'entrée	**Projection intermédiaire**	**Artefacts générés**
Activité BPMN	Activité du *runtime contract*	Tâche runtime, entrée de catalogue, vue de traitement, endpoint de complétion.
Flux BPMN	Transitions, entrées et sorties par activité	Logique d'orchestration, transitions d'instance, timeline.
Rôles candidats	Politique d'affectation	Liste de tâches, assignation, filtres de visibilité et contrôles d'autorisation.
Sources d'entrée	Lignage des données	Résolution des données à l'exécution, pré-remplissage de formulaires.
Mappings de sortie	Stratégie de stockage	Mise à jour de l'instance, des données partagées ou des deux.
Règles de visibilité	Droits de lecture activité/donnée	Filtrage back-end et adaptation des vues front-end.
Données partagées	Entités, champs et relations	Modèles persistants, catalogues, changelogs et métadonnées de formulaires.

3.5 Règles de transformation et génération des artefacts

Les règles de transformation déterminent la manière dont les modèles d'entrée deviennent des artefacts applicatifs. Dans ProOWeb, cette transformation suit deux étapes principales : la transformation modèle-à-modèle et la transformation modèle-à-texte. La première construit des contrats intermédiaires. La seconde produit les fichiers du projet cible.

Ce découpage est conforme à l'esprit de l'ingénierie dirigée par les modèles. Il évite de passer directement du BPMN vers le code, ce qui rendrait les règles difficiles à vérifier. Il permet également de tester séparément les contrats avant la génération finale. Une erreur dans une stratégie d'affectation ou dans un mapping de données peut ainsi être détectée avant qu'elle ne se transforme en défaut dispersé dans plusieurs fichiers.

3.5.1 Transformation modèle-à-modèle

La transformation modèle-à-modèle prend en entrée le BPMN et *spec-v1*. Elle commence par normaliser les activités. Chaque activité reçoit un type, des rôles candidats, une politique d'affectation, des entrées, des sorties et une visibilité. Les activités automatiques reçoivent en plus un gestionnaire, un type de tâche automatique, un mode de déclenchement et une configuration.

La transformation construit ensuite les flux. À partir du BPMN, ProOWeb détermine les transitions entre activités, les liens entrants et les liens sortants. Les activités de départ peuvent être configurées explicitement ou inférées à partir du graphe. Cette étape permet au runtime de savoir comment démarrer une instance et comment la faire progresser.

La transformation modèle-à-modèle construit également les données. Les mappings d'entrée et de sortie sont parcourus pour créer des arêtes de lignage. Les entités partagées sont collectées, soit parce qu'elles sont explicitement modélisées dans *sharedData*, soit parce qu'elles sont inférées à partir des chemins de données. Les avertissements produits à ce niveau permettent d'identifier les entités consommées ou produites qui ne sont pas complètement modélisées.

3.5.2 Construction du modèle applicatif intermédiaire

Le modèle applicatif intermédiaire est constitué des contrats utilisés par la génération. Il transforme les informations de haut niveau en structures proches de l'application cible. Le *runtime contract* porte l'exécution. Le *data contract* porte les données. Les catalogues générés préparent la consommation par le front-end et le back-end.

Ce modèle intermédiaire contient des résumés utiles. Le contrat d'exécution indique par exemple le nombre d'activités, les activités manuelles et automatiques, les transitions

et les rôles. Le contrat de données indique le nombre de sources d'entrée, de mappings, d'arêtes de lignage, d'entités partagées et d'avertissements. Ces résumés facilitent la validation, la simulation et la promotion.

La construction du modèle intermédiaire a également un rôle de stabilisation. Les contrats sont indépendants de la manière dont le BPMN est affiché dans l'éditeur. Ils sont aussi indépendants des templates exacts. Ils servent de frontière entre l'analyse du modèle et la production du code.

3.5.3 Transformation modèle-à-texte

La transformation modèle-à-texte produit les fichiers du projet. Elle s'appuie sur des templates. Chaque template correspond à un fichier ou à une famille de fichiers. Les templates back-end génèrent des classes Java, des fichiers Maven, des contrôleurs, des services, des ports, des adaptateurs, des fichiers de configuration et des tests. Les templates front-end génèrent des modules JavaScript ou JSX, des catalogues et des composants d'interface.

La transformation modèle-à-texte applique aussi les règles de gouvernance des templates. Si des overrides sont déclarés dans `.prooweb/template-overrides.json`, ils peuvent remplacer, préfixer, suffixer ou modifier un bloc dans le contenu généré. Cette personnalisation est appliquée de manière déterministe, selon la priorité et l'identifiant de l'override. Elle permet de conserver des adaptations durables sans modifier directement le code du générateur.

Chaque fichier généré est enregistré dans le manifeste des fichiers gérés. Le manifeste contient le chemin, le hash, les propriétaires et la catégorie. Cette information est indispensable pour la migration intelligente, car elle permet de comparer l'état attendu du fichier avec l'état réel dans le projet.

3.5.4 Règles de projection vers les artefacts générés

Les règles de projection associent les concepts du modèle enrichi à des artefacts techniques. Une activité manuelle est projetée vers une tâche runtime, une entrée dans la liste de tâches, un formulaire et une action de complétion. Une activité automatique est projetée vers un handler ou un service. Une transition est projetée vers une logique d'avancement. Une règle de visibilité est projetée vers un filtre back-end et une adaptation front-end.

Les données sont projetées selon leur stratégie de stockage. Lorsque la sortie d'une activité est stockée dans l'instance, elle alimente l'état de l'exécution. Lorsqu'elle est stockée dans les données partagées, elle peut être persistée comme information réutilisable. Lorsqu'elle utilise `BOTH`, elle participe aux deux espaces.

Les rôles sont projetés vers le contrôle d'accès. Les rôles candidats orientent les tâches. Les rôles de supervision gouvernent les opérations sensibles. Les rôles de visibilité filtrent

les informations retournées par les APIs. Cette projection assure la continuité entre la spécification et le comportement de l'application.

3.5.5 Algorithme général de génération

L'algorithme général de génération décrit le déroulement logique de ProOWeb. Il ne représente pas toutes les instructions internes, mais il montre les étapes principales nécessaires pour obtenir une application à partir du double modèle d'entrée.

Algorithm 1 Génération d'une application web orientée processus dans ProOWeb

Require : Modèle BPMN M_{bpmn}, spécification enrichie S_{v1}, configuration du projet C

Ensure : Application web générée A

1 : Lire et normaliser le modèle BPMN M_{bpmn}
2 : Extraire les activités, événements, passerelles et flux utiles
3 : Valider la cohérence entre les activités BPMN et les activités décrites dans S_{v1}
4 : Normaliser les rôles, affectations, entrées, sorties, visibilités et données partagées
5 : Construire le contrat d'exécution $C_{runtime}$
6 : Construire le contrat de données C_{data}
7 : Générer les catalogues de processus, de tâches, de formulaires et de lignage
8 : Appliquer les règles de projection vers les modules back-end
9 : Appliquer les règles de projection vers les modules front-end
10 : Appliquer les règles de personnalisation déclarées dans les overrides de templates
11 : Écrire les fichiers générés et calculer leurs empreintes
12 : Mettre à jour le manifeste des fichiers gérés
13 : Produire le rapport de génération ou de migration
14 : **return** Application web générée A

Cet algorithme met en évidence la place de la validation et des contrats. La génération intervient après la normalisation et la construction du modèle pivot. Cette organisation rend le résultat plus fiable que dans une approche où les fichiers seraient écrits directement depuis le diagramme.

3.6 Préservation du code personnalisé et synchronisation

La génération automatique devient réellement utile lorsque l'application peut évoluer. Dans un projet réel, les développeurs modifient le code, ajoutent des règles particulières, adaptent les interfaces et corrigent certains comportements. Si chaque nouvelle génération écrase ces modifications, le framework devient difficile à utiliser dans la durée. ProOWeb

répond à ce problème par un contrat de propriété du code, un manifeste des fichiers gérés, une migration intelligente et une gouvernance des templates.

Cette partie constitue un point important de la contribution. Le chapitre 2 a montré que plusieurs approches produisent des artefacts utiles, mais que la gestion conjointe du code généré et du code écrit manuellement reste une difficulté récurrente [20]. ProOWeb prend en compte cette difficulté dès l'architecture de génération.

3.6.1 Problème de la régénération

La régénération apparaît dès qu'une configuration change, qu'un pack fonctionnel est activé, qu'un processus est redéployé ou qu'une version du modèle évolue. Elle est nécessaire, car l'application doit rester alignée sur les modèles. Elle est aussi risquée, car elle peut remplacer des fichiers que des développeurs ont modifiés.

Le problème ne se limite pas à la perte de code. Une régénération non contrôlée peut rendre le projet instable : conflits de fichiers, incohérences entre back-end et front-end, disparition de règles spécifiques, changement non documenté dans la sécurité ou rupture des tests. Il faut donc traiter la régénération comme une opération d'ingénierie, et non comme un simple écrasement de répertoire.

ProOWeb distingue les fichiers gérés et les fichiers non gérés. Les fichiers gérés sont produits par le générateur et suivis dans `.prooweb-managed.json`. Les fichiers non gérés restent hors du contrôle de ProOWeb et ne doivent pas être modifiés par la génération. Lorsqu'un fichier géré a été modifié manuellement, la migration intelligente détecte la différence et crée une sauvegarde avant remplacement.

3.6.2 Préservation du code côté back-end

Côté back-end, le code généré est particulièrement sensible, car il contient les règles d'API, de sécurité, de persistance et de runtime. ProOWeb limite le risque en produisant une architecture modulaire où les responsabilités sont séparées. Les développeurs peuvent ajouter des extensions dans les modules adaptés, au lieu de modifier directement les fichiers purement générés.

Le manifeste permet de savoir quels fichiers appartiennent au générateur. Lors d'une migration, ProOWeb compare les empreintes. Si le fichier correspond à la version précédente, il peut être remplacé directement. Si le fichier a été modifié, il est sauvegardé dans `.prooweb/backups/` avant écriture. Le rapport de migration indique ensuite les fichiers créés, mis à jour, inchangés, sauvegardés ou concernés par des conflits résolus.

La préservation back-end repose aussi sur les templates overrides. Une équipe peut décider qu'un fichier doit recevoir un ajout systématique, un bloc personnalisé ou une modification contrôlée. Plutôt que de modifier manuellement chaque version générée, elle déclare une règle d'override. Cette règle est réappliquée lors de la génération, de la migra-

tion et du déploiement des processus.

3.6.3 Préservation du code côté front-end

Côté front-end, la personnalisation concerne souvent l'ergonomie, les composants visuels, les libellés, les parcours utilisateur et certaines intégrations. Ces modifications peuvent être nombreuses. Elles doivent donc être encadrées pour ne pas disparaître lors du redéploiement d'un processus.

ProOWeb génère des modules React organisés autour des processus et des catalogues. Les composants produits peuvent être complétés par des modules propres au projet. Les overrides permettent également d'adapter certains fichiers générés. Les stratégies disponibles sont `replace`, `prepend`, `append` et `replace-block`. La stratégie `replace-block` est particulièrement utile lorsque l'équipe veut remplacer une portion précise de contenu sans reprendre tout le fichier.

Comme pour le back-end, la migration produit un rapport. Ce rapport indique les fichiers concernés par des overrides, les applications réussies et les overrides ignorés. Cette transparence est importante, car une personnalisation non appliquée peut modifier le comportement de l'application sans erreur visible immédiate.

3.6.4 Cycle de synchronisation itératif

Le cycle de synchronisation itératif décrit la manière dont ProOWeb accompagne l'évolution du projet. Une première génération produit l'application de base. Les développeurs peuvent ensuite modifier certains fichiers, ajouter des extensions ou déclarer des overrides. Lorsque la configuration ou le processus évolue, ProOWeb reconstruit un snapshot de génération, compare ce snapshot avec l'état du projet, applique les règles de migration et produit un rapport.

Ce cycle transforme la génération en activité continue. L'application n'est pas seulement créée une fois ; elle peut être régénérée, migrée et adaptée. La synchronisation repose sur trois traces : le manifeste des fichiers gérés, les sauvegardes créées lors des conflits et les rapports de migration. Ces éléments permettent de comprendre ce qui a changé et de revenir sur une modification si nécessaire.

TABLE 3.5 – Mécanismes de préservation et de synchronisation dans ProOWeb

Mécanisme	**Rôle**	**Effet attendu**
Manifeste `.prooweb-managed.json`	Suivre les fichiers produits, leurs empreintes, leurs propriétaires et leurs catégories.	Identifier les fichiers que ProOWeb peut gérer lors d'une régénération.
Smart migration	Comparer l'état attendu et l'état réel, détecter les modifications manuelles et produire un rapport.	Éviter l'écrasement silencieux du code personnalisé.
Sauvegardes automatiques	Copier les fichiers conflictuels avant écriture dans `.prooweb/backups/`.	Permettre la récupération des adaptations manuelles.
Template overrides	Appliquer des personnalisations durables selon des stratégies déclarées.	Réduire les modifications manuelles répétées sur les fichiers générés.
Rapports de migration	Décrire les créations, mises à jour, conflits, collisions, sauvegardes et overrides.	Rendre l'évolution auditable et compréhensible.

La synchronisation itérative donne à ProOWeb une orientation différente d'un générateur ponctuel. L'objectif n'est pas seulement de produire beaucoup de fichiers au départ. Il s'agit de maintenir une relation durable entre les modèles, le code généré et le code personnalisé.

3.7 Conclusion

Ce chapitre a présenté l'approche proposée pour la génération d'applications web pilotées par BPMN. L'approche repose sur un double modèle d'entrée : le modèle BPMN décrit le workflow, tandis que la spécification *spec-v1* complète les informations nécessaires à l'application. Ces deux sources sont transformées en un modèle pivot composé de contrats intermédiaires, puis projetées vers des artefacts back-end, front-end, de persistance, de sécurité et de déploiement.

L'architecture de ProOWeb montre que la génération ne se limite pas à la production d'écrans ou de contrôleurs isolés. Elle vise une application web orientée processus, structurée autour d'un back-end Spring Boot modulaire, d'un front-end React/Vite, d'une gestion des données, d'un runtime de processus, de mécanismes d'affectation, de supervision et d'audit. Le framework cherche ainsi à combler l'écart entre le modèle de processus

et l'application réellement exploitable.

La contribution du chapitre réside aussi dans la prise en compte de l'évolution. Le code généré reste propriété de l'équipe de développement. ProOWeb suit les fichiers gérés, détecte les modifications manuelles, crée des sauvegardes, applique des overrides de templates et produit des rapports de migration. Cette gouvernance répond au problème de la régénération, souvent négligé dans les approches de génération.

Le chapitre suivant pourra s'appuyer sur cette approche pour présenter la conception technique, l'implémentation du framework et l'évaluation de la génération sur un cas d'utilisation représentatif.